Tip-of-the-Tongue Retrieval: A Multi-Stage Hybrid Retrieval Pipeline with a Single-Turn LLM-Based Query Reformulation

Debayan Mukhopadhyay
University of Calcutta
debayan.mukherjee14@gmail.com

Utshab Kumar Ghosh
Missouri University of Science and Technology
u.ghosh@mst.edu

Shubham Chatterjee
Missouri University of Science and Technology
shubham.chatterjee@mst.edu

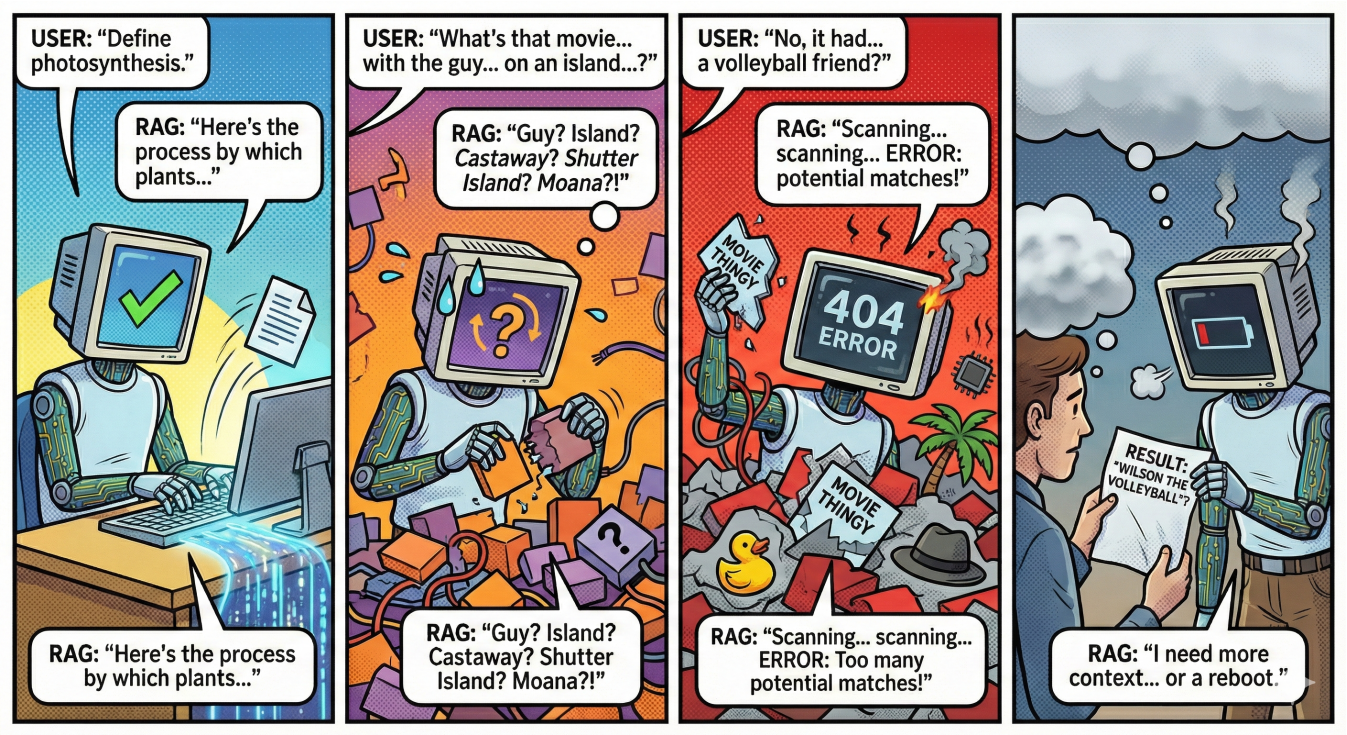

Figure 1: Demonstrates how traditional RAG systems trained on crisp keyword dense queries struggle in a ToT artifact laden setup. Illustration generated by Google AI Studio's Nano Banana Pro.

Abstract

Retrieving known items from vague, partial, or inaccurate descriptions, a phenomenon known as Tip-of-the-Tongue (ToT) retrieval, remains a significant challenge for modern information retrieval systems. In this work, we propose and evaluate a robust multi-stage retrieval pipeline designed specifically for the ToT domain. Our approach integrates Large Language Model (LLM) based query rewriting to bridge the gap between a well-formulated query and one laden with ToT artifacts. These rewritten queries are then fed into a multi-stage retrieval pipeline comprising a sparse retrieval stage (BM25), followed by a bi-encoder ensemble (ColBERTv2, Contriever, E5-large-base), a cross-encoder re-ranking (monoT5), and a final list-wise re-ranking stage using a 72B parameter LLM (Qwen 2.5 Instruct in 4-bit quantized mode). Through rigorous sequential tuning on the datasets provided in the 2025 TREC-ToT track, we demonstrate that our pipeline achieves state-of-the-art performance, with significant gains in both Recall and Normalized Discounted Cumulative Gain (nDCG). We analyze the contribution of each stage, highlighting the critical role of query rewriting and the strategies used to

,

direct the design choices that went behind the construction of the multi-stage pipeline.

1 Introduction

Tip-of-the-Tongue (ToT) refers to the process of retrieving information from fragmented, distorted, or incomplete memory. One of the most persistent and difficult problems in cognitive science and information retrieval (IR) is this phenomenon. ToT scenarios are distinguished by a fundamental disconnect between the user's internal representation of an item and the external identifiers needed to locate it, in contrast to traditional known-item search, where a user may recall a title or author imprecisely but retains a clear semantic link to the target [1, 3]. This disconnect shows up in queries that are frequently verbose, full of phenomenological descriptions of the user's experience (such as "I watched this in my childhood" or "The cover was blue"), and filled with false memories or uncertainty markers [1, 10].

The field is at a turning point as we get together for the TREC 2026 Tip-of-the-Tongue Track. This track's development, from its original focus on movie identification to the addition of landmarks and celebrities to the more recent open-domain reasoning challenges, reflects the larger evolution of artificial intelligence from pattern matching to agentic reasoning. Early methods relied on dense embeddings to bridge the lexical gap between a user's ambiguous description and a document's metadata, treating ToT retrieval as a specialized case of ad-hoc retrieval [2]. However, recent scholarship, particularly the introduction of benchmarks such as BLUR (Browsing Lost Unformed Recollections), has demonstrated that semantic similarity alone is insufficient for resolving the most difficult ToT queries [15]. Humans solve these problems not simply by matching terms, but by executing complex multi-hop reasoning chains, by validating hypotheses, rejecting false premises and utilizing external tools to bridge gaps in memory [15].

This document summarizes the theoretical foundations and methodological developments that characterize the current state of the art and functions as the foundational Notebook Paper for the 2026 track. We contend that ToT retrieval is no longer just a ranking problem. Rather, it needs to be rethought as an agentic, iterative process of *recollection reconstruction*, in which the retrieval system collaborates with the user to improve their memory trace.

The magnitude of the issue in practical contexts emphasizes how urgent this research is. Analyses of online communities such as Reddit `/tipofmytongue` reveal millions of unmet information needs and users often resort to human crowdsourcing only when traditional search engines have failed [2, 5]. In both personal and professional contexts, the persistent ToT state is associated with high levels of frustration and cognitive load [1]. In professional settings, the inability to find information again leads to significant productivity losses. Additionally, the democratization of query elicitation through Large Language Models (LLMs) [7] has enabled the creation of large synthetic datasets that can replicate user behavior at previously unachievable scales.

In this introduction, we describe how the ToT problem has persisted in spite of developments in large language models. Although models such as GPT-4 [12] and Claude 3.5 Sonnet show remarkable encyclopedic recall, they often experience hallucinations or are unable to navigate the "uncertainty regions" present in ToT queries [15]. In order to assess the *reasoning traces* and *tool-use capabilities* of systems, the 2026 track goes beyond static retrieval effectiveness. Our goal is to create systems that not only retrieve but also *deduce* by combining the most recent architectural advancements in agentic search with insights from cognitive psychology regarding memory access [3].

2 Literature Review

With concepts from information retrieval, natural language processing, cognitive psychology and library science, the literature on tip-of-the-tongue retrieval is inherently interdisciplinary. The cognitive underpinnings of the phenomenon, the emergence of community-based datasets, the development of specialized retrieval methodologies (from decomposition to simulation), and the current shift toward agentic reasoning benchmarks are the four distinct phases into which this review divides the landscape.

The theoretical basis for the ToT track is rooted in the seminal work of Brown and McNeill [3], who first experimentally induced and described the "Tip of the Tongue" state. They characterized it not as a state of total ignorance, but as a state of *partial access*. Subjects in a ToT state often possessed specific, albeit incomplete, information about the target word, such as its first letter, the number of syllables, or its stress pattern [3]. This finding is crucial for IR because it suggests that the "missing" query terms are not randomly absent; rather, users retain structural and sensory fragments of the target while losing the identifying label.

In the context of known-item retrieval, this translates to users recalling "peripheral" attributes rather than "central" identifiers. Arguello et al. expanded this cognitive definition into the digital search domain, conducting a comprehensive annotation study of ToT queries for movies [1]. They identified that searchers rely heavily on three types of information:

(1) **Content details:** Descriptions of plot, characters, or scenes.

(2) **Context of engagement:** Biographical details about when and where the user consumed the content (e.g. I watched this with my father in the 1990s).
(3) **Previous search attempts:** Meta-commentary on failed strategies (e.g. I already searched for "movie with red car" but found nothing).

Some queries are marked by linguistic hedging. Phrases like "I think", "maybe" or "it might have been" explicitly signal low confidence in specific query facets [1, 7]. This uncertainty brings a severe challenge for traditional lexical models like BM25, which treat all query terms with equal weight. Let a user query be "I think the actor was Tom Hanks". In this case, a standard retriever will heavily penalize documents without "Tom Hanks", even if the user's memory is incorrect and the actor was actually Bill Paxton. This phenomenon of "false memories" is a pervasive feature of ToT queries and requires systems that can treat query terms as probabilistic constraints rather than absolute requirements [6]. The linguistic structure of these queries further complicates retrieval.

Unlike the terse keywords typical of navigational search, ToT queries are notoriously verbose. Analysis of the *WhatsThatBook* dataset reveals an average query length of 156.20 words, vastly exceeding the 7-20 word averages found in standard retrieval benchmarks like MS MARCO or Natural Questions [10]. This verbosity dilutes the signal of key terms, as users often bury critical details within lengthy narratives about their search process or emotional state.

2.1 The Evolution of Datasets

Progress in ToT retrieval has been historically bottlenecked by data scarcity. Unlike ad-hoc retrieval, where relevance judgments can be crowd-sourced, ToT queries require a ground truth that only the original searcher can verify. The evolution of datasets reflects a migration from small, hand-curated collections to massive, community-mined and synthetically generated corpora.

2.1.1 ***Community Question Answering (CQA) :.*** Mining Community Question Answering (CQA) platforms, particularly the `r/tipofmytongue` subreddit, led to the first breakthrough in data availability.

- **Reddit-TOMT:** This method was formalized by Bhargav et al., who created the Reddit-TOMT dataset [2] by scraping solved threads from Reddit. 15,000 query-item pairs with a movie and book focus were included in this dataset. It determined the task's baseline difficulty and demonstrated that neural retrievers such as DPR (Dense Passage Retrieval) greatly outperformed lexical baselines because of the semantic difference between the formal metadata of the items and the user's verbose description [2].
- **TOMT-KIS:** This method was greatly expanded by Frobe et al., who published the TOMT-KIS dataset with 1.28 million known-item questions [5]. Only 47% of the queries had identified answers and the emphasis was primarily on Query Performance Prediction (QPP) rather than pure retrieval effectiveness, despite the dataset's unmatched scale. By using the term "time to solve" as a stand-in for query difficulty, their analysis showed that known-item questions have different performance characteristics than typical informational queries [5].
- **WhatsThatBook:** Further specializing, Lin et al. presented the WhatsThatBook dataset, which consists of 14,441 pairs created especially for book retrieval [10]. This dataset was special because it included rich metadata fields, such as images of book covers, which made multimodal retrieval techniques possible. It highlighted that users often recall visual features ("a yellow cover with a cat silhouette") that textual descriptions in standard indices fail to capture [10].

2.1.2 ***The Synthetic Turn: Elicitation and Simulation :*** Despite the volume of CQA data, these datasets suffer from inherent domain skew. As noted by He et al., CQA platforms are heavily biased toward casual leisure topics such as movies, games and music, leaving domains like "People" or "Landmarks" underrepresented [7]. To address this, the field has moved toward *query elicitation* and *simulation*.

He et al. introduced a dual-method framework for generating ToT queries [7]. First, they employed **LLM-based Simulation**. By prompting Large Language Models (LLMs) with Wikipedia summaries and specific instructions to "role-play" a forgetful user (e.g. "express doubt", "mix up details"), they generated synthetic queries that achieved high correlation with real human queries in terms of system ranking performance. Second, they utilized **Human Elicitation via Visual Stimuli**. To capture genuine human memory failures, they developed an interface that showed participants images of famous entities (movies, landmarks) and asked them to recall the name. If the participant recognized the entity but failed to name it, they were in a genuine ToT state and were asked to write a query.

2.2 Methodological Advances: Decomposition and Multimodality

The complexity of ToT queries renders standard single-vector retrieval ineffective. A single embedding vector struggles to encode the "plot", "cover description", "time period" and "user sentiment" simultaneously without losing fidelity. This has led to the adoption of *Query Decomposition* and *Facet-Specific Retrieval*.

2.2.1 ***Query Decomposition :*** Lin et al. proposed a framework that decomposes complex ToT queries into individual "clues" or sub-queries, which are then routed to specialized retrieval experts [10]. For instance, a query mentioning "a yellow cover" and "a plot about a boy and a cat" is split. A "Cover Retriever" (using CLIP) processes the visual description, while a "Plot Retriever" (using Contriever) processes the narrative elements.

They distinguished between **Extractive** and **Predictive** decomposition. Their results showed that **Predictive Decomposition** significantly outperformed extractive methods, as it bridges the gap between the user's *experience* (context) and the document's *metadata* (facts) [10].

2.2.2 ***Multimodal Integration :*** The integration of visual data has become increasingly central. The *WhatsThatBook* dataset demonstrated that cover image retrieval (via CLIP) could resolve queries that were textually ambiguous [10]. Similarly, the BLUR benchmark contains file uploads (images, audio clips) as query inputs, which shows the reality that users often possess a visual fragment (e.g. a screenshot of a scene) but lack the semantic label [15].

2.3 The Agentic Shift: Reasoning and Benchmark Validity

The most significant recent development, driving the agenda for TREC 2026, is the realization that ToT retrieval is often a **multi-hop reasoning problem** rather than a static matching problem. Wang et al. introduced the **BLUR (Browsing Lost Unformed Recollections)** benchmark to test this capability rigorously [15].

2.3.1 ***Limitations of Static Retrieval :*** BLUR showed that even the most advanced static retrievers and base LLMs (such as GPT-4o [11] or Llama 3.1) fall well short of near-perfect human performance (98%) on complex ToT questions, achieving only ∼56% accuracy [15]. Models have trouble with **orchestration** and **contextual verification**, according to the failure case analysis.

2.3.2 ***Tool Use and Agentic Systems :*** A change toward **Agentic Search** has been required as a result. According to this paradigm, the system is an active agent capable of using tools (browsers, maps, OCR, calculators) rather than a passive retriever. The BLUR benchmark specifically assesses how well a system can plan a search strategy, carry out actions and compare the results to the user's ambiguous description [15]. However, Wang et al. discovered an unexpected "Agentic Gap" - complex agent frameworks frequently only slightly outperformed the base LLMs they wrapped [15]. This implies that the fundamental challenge is not only having access to tools but also having the **reasoning capability** needed to deal with uncertainty.

3 Methodology

3.1 Motivation: The Challenge of ToT Artifacts

Our initial analysis of the queries provided by the TREC-TOT 2025 Track Organizers [14] revealed significant challenges for current State-of-the-Art (SoTA) Retrieval-Augmented Generation (RAG) systems. The original queries often contain what we term "ToT artifacts", which are verbose, circumlocutory descriptions characteristic of the Tip-of-the-Tongue phenomenon, where the user describes an item without naming it directly. These artifacts introduce noise that traditional sparse retrievers like BM25 struggle to handle effectively.

We established this finding quantitatively by analyzing the baseline performance. As shown in (Table **??**), the original queries yield suboptimal retrieval performance across all dataset splits. For instance, on the `dev2` set, the original queries achieved an nDCG@10 of only 0.129 with BM25. This motivated our adoption of a query rewriting mechanism to transform these artifact-laden descriptions into more precise, keyword-centric queries suitable for efficient retrieval.

3.2 Multi-Model Query Rewriting

To mitigate the impact of ToT artifacts, we employed a diverse rewriting strategy leveraging three distinct Large Language Models (LLMs): **Meta-Llama-3.1-8B-Instruct** [4], **Mistral-7B-Instruct-v0.3** [9], and **Qwen2.5-7B-Instruct** [13]. To ensure computational efficiency while maintaining generation quality, all models were deployed in 8-bit quantized modes.

The core of our strategy involves directing these LLMs to strip away the conversational fluff and focus on the semantic core of the user's intent. We utilized the following specific prompt to guide the rewriting process:

> **System Prompt:** "You are a query rewriter for a search engine. Your task is to rewrite a complex, verbose, tip-of-the-tongue description into a simple, keyword-focused search query.
>
> Guidelines: 1. Identify the core entity type (e.g., movie, book, song) if implied.
> 2. Extract key details: plot points, character names, actors, famous scenes, or unique identifiers.
> 3. Remove conversational filler ("I think it was...", "It might be...", "I remember seeing...").
> 4. Remove negative constraints or uncertainty unless crucial ("Not sure if...").
> 5. Formulate a concise query that a standard search engine (like Google or BM25) would understand.
> 6. Output ONLY the rewritten query text. Do not output any explanations."

This prompt forces the models to act as strict filters, distilling the user's vague recollection into a structured search query. **(Figure 2)** illustrates this parallel processing pipeline.

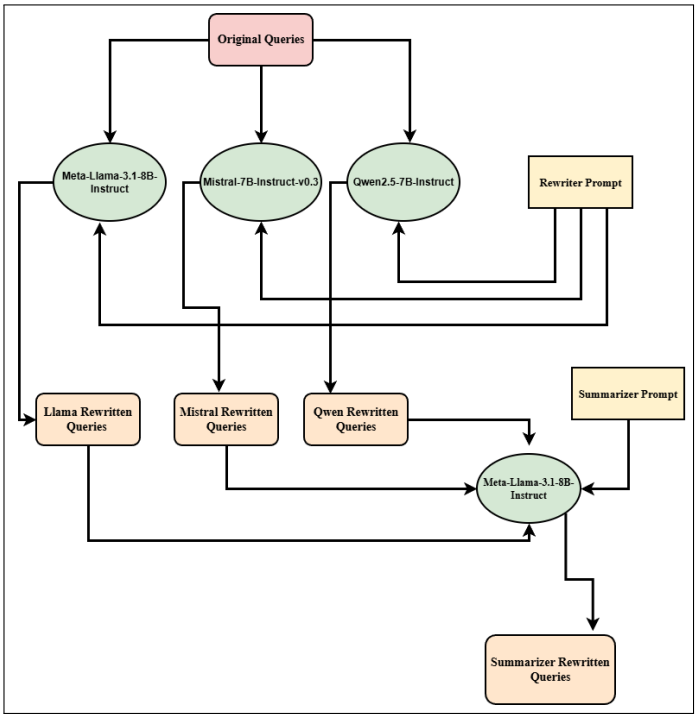

Figure 2: Flowchart of the Query Rewriting and Summarization Pipeline. Original queries are processed in parallel by three LLMs, and their outputs are fused by a summarizer.

3.3 Query Summarization and Fusion

While the individual rewriters provide distinct perspectives, we hypothesized that a consensus view would offer the most robust query representation. To achieve this, we introduced a fourth stage: a **Summarizer** utilizing the **Meta-Llama-3.1-8B-Instruct** model.

This summarizer takes the three rewritten variations as input and fuses them into a single, high-quality query. This step is designed to preserve the most salient keywords identified by the ensemble while discarding hallucinations or idiosyncratic errors from any single model. The summarization was driven by the following prompt:

> **System Prompt:** "You are an expert search query optimizer. You will be given three different rewrites of the same original vague query. Your goal is to synthesize the BEST possible search query by combining the unique information from all three sources.
>
> Instructions:
> 1. Analyze the three rewrites for common keywords and unique, high-value details (e.g., names, specific actions).
> 2. Create a single, consolidated query that captures the maximum amount of relevant information.
> 3. Keep it keyword-rich and concise.
> 4. Output ONLY the final summarized query."

3.4 Sparse Retrieval (Stage 1)

To establish a strong sparse retrieval baseline, we employed two standard probabilistic models: Okapi BM25 and the Divergence from Randomness model PL2. We implemented a rigorous, sequential hyperparameter optimization strategy to maximize retrieval performance, specifically targeting nDCG@10.

3.4.1 ***Sequential Adaptive Grid Search :*** We adopted a multi-stage tuning approach that progressed sequentially through our dataset splits (`train` → `dev1` → `dev2`). This strategy allows for the refinement of parameters on unseen data while carrying forward the "knowledge" (optimal parameters) from the previous stage, thereby preventing overfitting to a single development set.

The optimization process was divided into two distinct phases:

1. Phase I: Global Search (Training Set) : On the initial training set, we conducted a coarse-grained grid search over a broad range of values to identify the global optimal region. The search spaces were defined as follows:

- **BM25:** We explored $k_1 \in [0.4, 4.0]$ and $b \in [0.3, 1.0]$
- **PL2:** We explored the parameter $c \in [0.1, 20.0]$.

2. Phase II: Refined Local Search (Development Sets) : For subsequent datasets (`dev1`, `dev2`), we employed a refined local search strategy. Instead of searching the global grid again, we generated a narrower, high-resolution grid centered around the best parameters found in the previous stage. Let θ_{best} be the optimal value for a parameter from the previous dataset. The new search space S was constructed as:

$$S = \text{linspace}(\theta_{best} - \delta, \theta_{best} + \delta, \text{steps} = 5) \quad (1)$$

where δ represents the refinement radius ($\delta_{k1} = 0.4$, $\delta_b = 0.2$, $\delta_c = 2.0$).

3.4.2 ***Implementation Details :*** The optimization pipeline was implemented using the PyTerrier framework. To handle the computational load of evaluating hundreds of configurations over the 6.4M document corpus, we utilized a high-performance computing environment with 500GB of RAM. We employed parallel processing via `ThreadPoolExecutor`, allowing for concurrent evaluation of multiple parameter

configurations. The index structures were loaded into memory using the `fileinmem` property to minimize I/O latency during the iterative retrieval process.

3.5 Dense Retrieval Ensemble (Stage 2)

Following the initial sparse retrieval, we employed a dense retrieval stage to address the vocabulary mismatch inherent in Tip-of-the-Tongue queries. This stage was designed as a high-recall filter, aggregating semantic signals from multiple architectures.

3.5.1 ***Ensemble Architecture :*** We implemented a parallel ensemble of three distinct bi-encoder models to capture diverse semantic features:

(1) **ColBERTv2.0:** A late-interaction model that retains token-level granularity, effective for capturing fine-grained details in verbose ToT queries.
(2) **Contriever:** An unsupervised dense retriever trained via contrastive learning [8], offering robust zero-shot performance.
(3) **E5-Large:** A text-embedding model trained with weak supervision, providing strong semantic matching capabilities.

The outputs of these three models were combined using Reciprocal Rank Fusion (RRF) with a constant $k = 60$ to produce a single, robust candidate list.

3.5.2 ***Robust Candidate Depth Tuning :*** A critical design choice was the selection of the candidate depth K_{dense}. Unlike standard approaches that optimize for nDCG, we optimized for **Recall** to ensure the relevant document was retained for downstream stages. To avoid overfitting to the training set, we employed a *Robust Aggregate Tuning* strategy. We defined a robust score S_K for each candidate depth K:

$$S_K = \mu(R@K) - \alpha \cdot \sigma(R@K) \tag{2}$$

where μ and σ are the mean and standard deviation of Recall@K across the `train`, `dev1`, and `dev2` splits, and $\alpha = 0.5$ is a stability penalty.

To implement this efficiently, we utilized a "Virtual Tuning" approach: we ran inference once to generate a master list of the top-5000 candidates and subsequently simulated smaller K values by slicing this list.

3.6 Cross-Encoder Re-ranking (Stage 3)

The dense retrieval stage, while improving recall, introduced precision errors by prioritizing broad semantic matches over specific keyword constraints. To correct this "semantic drift", we implemented a Cross-Encoder stage using the `monoT5-large` model.

3.6.1 ***Hybrid Input Pooling :*** To maximize the probability of including the relevant document, we devised a hybrid input strategy. Rather than re-ranking only the dense output, we constructed a union pool consisting of the top-100 candidates from the sparse BM25 stage and the top-100 candidates from the dense RRF stage. This ensured that the Cross-Encoder had access to both exact keyword matches and semantic matches.

3.6.2 ***Implementation and Optimization :*** The `monoT5` model processes query-document pairs jointly, which is computationally intensive. We optimized throughput by enabling FP16 (half-precision) inference and distributing the workload across two NVIDIA RTX 6000 GPUs using `DataParallel`.

3.7 Large Language Model Re-ranking (Stage 4)

The final stage utilized a 72B parameter Large Language Model, `Qwen2.5-72B-Instruct` (4-bit quantized), to perform zero-shot listwise re-ranking. This stage served as the final arbiter, leveraging the LLM's world knowledge to resolve subtle ambiguities in ToT descriptions.

3.7.1 ***Listwise Ranking Strategy :*** We employed a listwise approach, in which the LLM is presented with the query and a batch of candidates simultaneously and prompted to output the optimal permutation of Document IDs. This method allows the model to compare candidates directly against each other, rather than scoring them in isolation.
A manual inspection of Wikipedia pages revealed that, the Introduction section of these web pages are usually in the range 450-600 characters. To fit within the context window while retaining critical information, document text was truncated to the first 500 characters, capturing the introductory summaries typical of Wikipedia articles.
While it is highly likely that the queries might ask for information buried deeper inside the full text, supplying the full text as the context window has the following issues:

- **Cost/Speed:** Processing full documents (e.g., 2000 tokens) for 50 candidates would explode the input size to 100k+ tokens per query. This would make the 72B model extremely slow, essentially moving from seconds to minutes to process each query, and could also confuse it with irrelevant details.

- **"Lost in the Middle":** LLMs often focus on the beginning and end of long contexts. Truncating ensures the model focuses on the most "dense" information (the start). ToT queries are often descriptive and conceptual (e.g., "movie where guy is stuck on Mars growing potatoes"). The relevant document (e.g., a Wikipedia plot summary) usually contains the key matching concepts early in the text (title, introduction, first paragraph).

3.7.2 ***Joint Hyperparameter Tuning :*** Recognizing the dependency between the Cross-Encoder and the LLM, we performed a joint grid search to optimize the input depth (K_{cross}) fed to the LLM and the final re-ranking depth (K_{llm}).

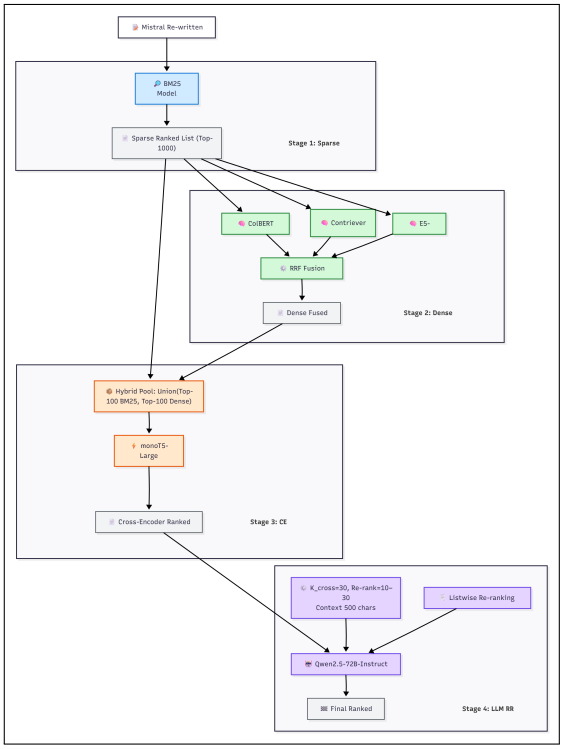

Figure 3: The proposed Retrieval Pipeline with four distinct stages: the Sparse Retrieval Stage (Stage 1), Dense Retrieval Bi-encoder (Stage 2), Cross-Encoder Stage (Stage 3), and the LLM Re-ranker Stage (Stage 4). The input re-written query set in the diagram has been generated using Mistral-7B-Instruct-v0.3, as discussed in Section 4.

4 Results and Analysis

4.1 Efficacy of LLM-based Query Rewriting

The introduction of an LLM-based rewriting step at the beginning of the retrieval pipeline introduces a non-trivial computational cost and latency overhead. Therefore, the primary question we must address is whether this additional complexity translates into justifiable performance gains.

To evaluate this, we analyze the performance of four distinct Large Language Models (LLMs)-Llama, Mistral, Qwen, and a single-turn Summarized approach, against the baseline of the original, raw Tip-of-the-Tongue (ToT) queries.

It is important to establish our evaluation protocol regarding the dataset splits. The `train`, `dev1`, and `dev2` splits were utilized exclusively to fine-tune the hyperparameters of the retrieval models (specifically $k1$ and b for BM25, and c for PL2), a process detailed in **(Section ??)**. Consequently, the `dev3` split functions as a pseudo-test set, representing unseen data where no parameter optimization was performed. This distinction ensures that the results presented on `dev3` reflect generalized performance.

4.1.1 ***Performance Comparison and Model Selection :*** The results on the pseudo-test set (`dev3`) clearly demonstrate that Query Rewriting provides a massive performance boost compared to the original queries. As illustrated in **(Figure 4)**, there is a substantial quantitative gap between the rewritten queries and the baseline.

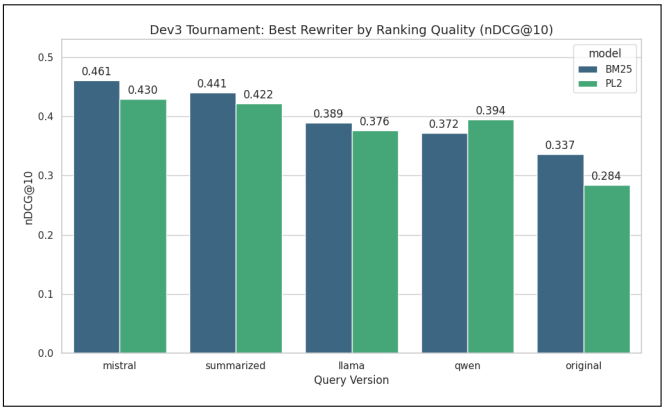

Figure 4: Comparative nDCG@10 scores of BM25 and PL2 retrieval models across different rewriter strategies.

(Table 1) quantifies this lift using the best performing hyperparameters on the `dev3` set. Mistral emerges as the clear winner, achieving the highest scores across the board.

Table 1: Performance Lift on Pseudo-Test Set (dev3) using BM25

Query Version	nDCG@10	Success@10	Lift (nDCG)
Mistral	**0.4609**	**0.5672**	+36.9%
Summarized	0.4406	0.5672	+30.9%
Llama	0.3889	0.5019	+15.5%
Qwen	0.3720	0.4795	+10.5%
Original	0.3366	0.4347	Baseline

These findings lead to several critical conclusions:

1. Rewriting is Essential : The baseline performance using original ToT queries creates a significant bottleneck. With an nDCG@10 of only 0.3366, relying on raw queries

Table 2: Performance Lift on Pseudo-Test Set (dev3) using PL2

Rewriter	Success@10	nDCG@10	Lift (nDCG)
Llama	0.4720	0.3760	+32.2%
Mistral	**0.5299**	**0.4299**	+51.2%
Qwen	0.4832	0.3945	+38.7%
Summarized	**0.5429**	0.4222	+48.4%
Original	0.3787	0.2844	Baseline

for the subsequent dense retrieval stage would severely limit potential recall. The 36.9

2. Mistral vs. Summarized Efficiency : While the Summarized approach (generating multiple queries and summarizing them) remains competitive, specifically tying Mistral on Success@10 (0.5672), it falls short on ranking quality (nDCG). From an efficiency standpoint, Mistral is the superior choice. The Summarized method requires inputs from 3 separate LLMs as discussed previously **??**. In contrast, Mistral achieves the state-of-the-art result in a single inference call, making it the most efficient high-performance candidate.

(Figure 5) further corroborates this, showing that Mistral consistently maximizes Success@10 across both BM25 and PL2 probabilistic models.

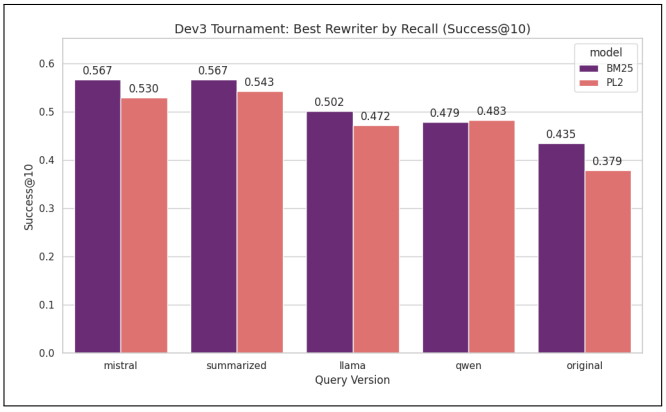

Figure 5: Success@10 performance of BM25 and PL2 retrieval models relative to rewriter selection.

4.1.2 ***Robustness Across Data Shifts :*** A detailed examination of the performance across different splits reveals a significant variance in absolute scores, highlighting the inherent complexity of the ToT domain. As observed in **(Figure 6)**, the nDCG@10 scores on dev3 (ranging 0.33–0.46) are significantly higher than those on dev2 (ranging 0.09–0.18).

This data shift implies that ToT queries come in a wide variety of shapes, sizes, and resolution difficulties. The dev2 split appears to contain highly ambiguous or lexically sparse

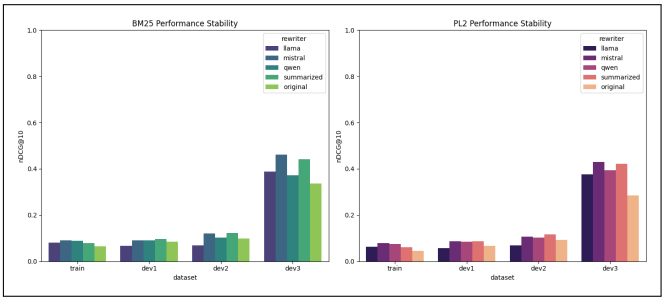

Figure 6: Stability of rewriter performance (nDCG@10) across four distinct dataset splits, illustrating the difficulty variance in ToT queries.

descriptions that challenge even the best rewriters, whereas dev3 likely contains examples with better query-document matching potential.

However, crucially, the *relative ranking* of the rewriters remains consistent regardless of the split difficulty. Mistral consistently outperforms the other LLMs and the baseline on the difficult splits (train, dev1, dev2) just as it does on the easier dev3 split. This confirms that the superiority of the Mistral-based rewriting approach is robust and not an artifact of a specific subset of easy queries.

4.2 Sparse Parameter Tuning

To ensure a fair evaluation, we optimized the hyperparameters for both the BM25 and PL2 retrieval models using a Sequential Adaptive Grid Search Strategy that maximizes performance efficiency. This has been thoroughly discussed in **??**

4.2.1 ***Optimization Analysis and Convergence :*** **BM25 Parameter Evolution:** The evolution of BM25 parameters for the Mistral-rewritten queries is visualized in **(Figure 7)**. We observed a distinct convergence toward a very low k_1 value (0.01). In the context of BM25, a low k_1 implies that the model saturates term frequency contributions rapidly-effectively treating term occurrence as binary. This indicates that for high-quality rewritten queries, the mere *presence* of the correct expansion terms is sufficient for retrieval, and additional frequency weight is unnecessary.

PL2 Parameter Evolution: Similarly, the PL2 optimization process **(Figure 8)** displayed robust stability. The global search on the training set identified a clear peak around $c = 2.5$. Subsequent refinement on the development splits maintained this optimal value, suggesting that the normalization characteristics required for the Divergence from Randomness model are consistent across the data distribution.

4.2.2 ***Optimized Performance Comparison :*** **(Table ??)** presents the final performance on the pseudo-test set (dev3)

using the optimal parameters derived from the sequential tuning process. Even when the Original queries are fully optimized (using their specific best parameters $k_1 = 0.01, b = 0.75$), they significantly lag behind the optimized Mistral queries.

The Mistral-based pipeline, tuned to $k_1 = 0.01$ and $b = 0.55$, achieves a dominant nDCG@10 of 0.4609, confirming that parameter tuning alone cannot bridge the vocabulary gap; it must be paired with generative rewriting to maximize efficacy.

Table 3: Final Tuned Performance on Pseudo-Test Set (dev3) - BM25

Query Type	Optimal Params	nDCG@10	Success@10
Mistral	$k_1 = 0.01$, $b = 0.55$	**0.4609**	**0.5672**
Original	$k_1 = 0.01$, $b = 0.75$	0.3366	0.4347

Table 4: Final Tuned Performance on Pseudo-Test Set (dev3) - PL2

Query Type	Optimal Params	nDCG@10	Success@10
Mistral	$c = 2.5$	**0.4299**	**0.5299**
Original	$c = 1.76$	0.2844	0.3787

4.3 Analysis of Rank Fusion Strategies

Following the optimization of individual sparse retrievers, we investigated whether combining the signals from BM25 and PL2 via Reciprocal Rank Fusion (RRF) could yield further performance gains. Theoretically, RRF thrives when fusing complementary lists where models retrieve different relevant documents. However, our empirical results strongly contraindicate this approach for the current pipeline.

4.3.1 ***The "Signal Dilution" Effect :*** As detailed in **(Table 5)** and 6 , the application of RRF resulted in a consistent performance regression compared to the single best model (Mistral BM25). Instead of additive gains, we observed a "signal dilution" effect. Since the Mistral-enhanced BM25 model significantly outperforms the PL2 model (as established in the previous section), fusing the two effectively pollutes the high-quality ranking of the former with the lower-quality signals of the latter.

Quantitatively, this degradation was substantial. On the training set, fusion caused a -10.8% drop in nDCG@10. Even on the pseudo-test set (dev3), where generalization is key, RRF resulted in a -3.0% loss in nDCG@10 and a -3.6% drop in Success@10.

Table 5: RRF Fusion Degradation vs. Mistral BM25 (nDCG@10)

Split	Mistral BM25	RRF Fusion	Gain/Loss
Train	0.0904	0.0807	-10.8%
Dev2	0.1198	0.1108	-7.5%
Dev3 (Test)	0.4609	0.4469	-3.0%

Table 6: RRF Fusion Degradation vs. Mistral BM25 (Success@10)

Split	Mistral BM25	RRF Fusion	Gain/Loss
Dev3 (Test)	0.5672	0.5466	-3.6%

4.3.2 ***Parameter Sensitivity and Strategic Decision :*** To ensure this result was not due to suboptimal hyperparameters, we conducted a grid search for the RRF constant k ranging from 10 to 100. As illustrated in **(Figure 10)**, the performance remained remarkably flat across the entire parameter space for a given split. This lack of sensitivity confirms that the performance ceiling is dictated by the retrieval lists themselves, not the fusion parameters.

Strategic Decision: Given that RRF introduces additional computational overhead (running two retrievers) while strictly degrading performance, we elected to drop the fusion module. Consequently, the optimized Mistral BM25 model is selected as the sole candidate for the subsequent Dense Retrieval and Reranking stages.

4.4 Analysis of Bi-Encoder Performance

Following the sparse retrieval stage, we employed a dense retrieval ensemble consisting of ColBERTv2.0, Contriever, and E5-large. These models were combined using Reciprocal Rank Fusion (RRF) with a constant $k = 60$.

To ensure computational efficiency without sacrificing coverage, we first performed the Robust Candidate Depth Tuning thoroughly discussed in **(Section 3.5.2)**.

4.4.1 ***Optimal Candidate Depth Selection :*** The depth tuning process revealed a decisive plateau in performance across all three dense models. As illustrated in **(Table 7)**, increasing the candidate depth (K) from 1,000 to 5,000 yielded zero improvement in the Robust Score or Mean Recall.

This phenomenon confirms that the "recall ceiling" is strictly determined by the top-1000 documents retrieved by the sparse stage. Consequently, we selected $K = 1000$ as the optimal depth. Fetching candidates beyond this point incurs computational cost with zero potential gain in relevant document coverage.

Table 7: Results of Robust Candidate Depth Tuning for ColBERT. The performance plateaus at K=1000, indicating that deeper sparse retrieval yields no added relevant documents.

Depth (K)	Mean Recall	Std Dev	Robust Score
500	0.4134	0.0335	0.3967
1000	**0.5022**	**0.0448**	**0.4798**
1500	0.5022	0.0448	0.4798
...	...	...	...
5000	0.5022	0.0448	0.4798

4.4.2 ***The Sparse vs. Dense Trade-off :*** Upon executing the bi-encoder stage with $K = 1000$ and fusing the results, we observed a specific phenomenon often described as the "Sparse Ceiling". **(Table 8)** compares the performance of the input list (Mistral BM25) against the output list (Dense Fusion) on the pseudo-test set.

Table 8: Performance comparison between the Input (Sparse) and Output (Dense Fusion) stages on the dev3 split.

Metric	Mistral BM25	Dense Fusion	Change
nDCG@10	**0.4609**	0.3778	-18.0%
Success@10	**0.5672**	0.5448	-3.9%
R@1000	0.8284	0.8284	**0.0%**

This data highlights a critical divergence between Ranking Quality and Recall Coverage:

(1) **Ranking Degradation:** The Dense stage significantly degraded the ranking quality, dropping nDCG@10 by 18%. This likely stems from the nature of the rewritten queries. Mistral optimized the ToT queries for lexical overlap (keywords), which BM25 exploits perfectly. The Dense models, trained on natural semantic matching, likely penalized these "keyword-stuffed" queries, down-ranking exact matches in favor of semantically broad but partially incorrect documents.

(2) **Recall Stability:** Crucially, the Recall@1000 remained identical at 0.8284. This confirms that **no relevant documents were lost** during the dense re-ranking; they were simply displaced from the top ranks to lower positions (likely ranks 11–100).

Strategic Decision: While the drop in nDCG is suboptimal, the preservation of Recall is the deciding factor. The subsequent Cross-Encoder stage is explicitly designed to repair ranking errors within a high-recall pool. By proceeding with the Dense Fusion lists, we aim to combine the semantic signals captured by the bi-encoders with the precision of the Cross-Encoder, relying on the fact that the relevant documents are still present within the candidate pool.

4.5 Analysis of Cross-Encoder Reranking

To address the ranking degradation observed in the dense retrieval stage, we deployed a Cross-Encoder (monoT5) to re-rank the fused candidate lists. Unlike bi-encoders, which rely on compressed vector representations, the cross-encoder processes the query and document simultaneously, allowing for a fine-grained assessment of semantic relevance.

4.5.1 ***Performance Recovery and State-of-the-Art Results :*** The results on the pseudo-test set (`dev3`) demonstrate a "Phoenix-like" recovery. As detailed in **(Table 9)**, the Cross-Encoder not only repaired the performance loss incurred during the dense stage but significantly outperformed the initial sparse baseline.

Table 9: Performance comparison between the Dense Baseline (Input) and Cross-Encoder (Output) on the dev3 split.

Metric	Dense Baseline	Cross-Encoder
nDCG@10	0.3778	**0.5196**
Success@10	0.5448	**0.6549**
Recall (Total)	0.8284	0.7332

Metric	Gain vs Bi-encoder Baseline
nDCG@10	+37.5%
Success@10	+20.2%
Recall (Total)	-11.5%

Metric	Gain vs Sparse Baseline
nDCG@10	+12.7%
Success@10	+15.5%
Recall (Total)	N/A

Two critical insights emerge from this data:

- **Correction of Semantic Confusion:** The Dense models struggled with the keyword-heavy nature of the rewritten queries, often burying relevant documents outside the top-10. The monoT5 model successfully identified these relevant pairs within the deeper candidate pool and elevated them to the top ranks, boosting Success@10 from 0.54 to 0.65.
- **Recall Trade-off:** The total recall dropped from 0.828 to 0.733. This is an expected consequence of truncating the list to a fixed depth for re-ranking. However, the trade-off is highly favorable: we sacrifice the long-tail recall (documents ranked >300) to secure a massive improvement in top-tier ranking quality.

4.5.2 ***Optimization of Re-ranking Depth :*** Given the high computational cost of Cross-Encoder inference, optimizing the candidate depth (K) is crucial for pipeline latency. We performed a robust tuning sweep to identify the smallest K that maximizes ranking quality without introducing instability.

As shown in **(Table 10)**, the robust score peaks at a remarkably shallow depth of $K = 50$.

Table 10: Robust tuning results for Cross-Encoder candidate depth.

Depth (K)	Mean nDCG	Std Dev	Robust Score
50	**0.2028**	0.0186	**0.1935**
100	0.1984	0.0184	0.1892
250	0.2009	0.0174	0.1922
300	0.1995	0.0180	0.1905

This finding indicates that the "signal" for the Cross-Encoder is concentrated in the very top portion of the Dense Fusion list. Re-ranking deeper candidates (100–300) introduces more noise than relevance, diluting the robust score. Consequently, we configured the final pipeline to re-rank only the top 50 candidates, ensuring maximum efficiency for the final LLM generation stage.

4.6 Analysis of List-wise LLM Re-ranking

To achieve the final tier of ranking precision, we employed Qwen-72B as a List-wise Re-ranker. Unlike the point-wise or pair-wise scoring of previous stages, the LLM evaluates the candidate list holistically, leveraging its broad world knowledge to resolve the complex descriptions typical of ToT queries.

4.6.1 ***State-of-the-Art Performance on Test Data :*** The deployment of the LLM reranker resulted in a dramatic performance leap on the blind test set (`dev3`), establishing a new state-of-the-art benchmark for this pipeline. As shown in **(Table 11)**, the LLM successfully refined the Cross-Encoder's output, achieving an nDCG@10 of **0.6347**.

Table 11: Performance comparison between the Cross-Encoder (Input) and LLM Reranker (Output) on the pseudo-test set (dev3).

Metric	Cross-Encoder	LLM Reranker	Gain
nDCG@10	0.5196	**0.6347**	+22.1%
Success@10	0.6549	**0.6828**	+4.3%
MRR	0.4808	**0.6213**	+29.2%

The **29.2% boost in MRR** is particularly telling. It indicates that Qwen-72B is exceptionally effective at identifying the ground truth within the top candidates and moving it to the Rank-1 position-a nuance that the smaller Cross-Encoder often missed.

4.6.2 ***Generalization vs. Overfitting :*** An analysis of the training and validation splits reveals an interesting contrast **(Table 12)**. While the model generalized exceptionally well to unseen data (`dev3`), it caused a regression on the training set.

Table 12: Impact of LLM Reranking across different dataset splits.

Dataset	CE Score	LLM Score	Gain/Loss
Train	0.2267	0.1956	-13.7%
Dev1	0.1815	0.1858	+2.4%
Dev2	0.2000	0.2167	+8.3%
Dev3 (Test)	**0.5196**	**0.6347**	**+22.1%**

This regression on `train` (-13.7%) is likely a byproduct of the pipeline optimization process. The previous stages (Sparse, Dense, Cross-Encoder) were heavily tuned on the training set, potentially "overfitting" the ranking to specific patterns in that data. The LLM, acting as a zero-shot generalist, re-ordered these lists based on broader semantic reasoning. The fact that performance improved significantly on all validation/test splits confirms that the LLM's ranking is more robust and generalizable than the tuned pipeline's output.

4.6.3 ***Joint Parameter Tuning :*** To optimize efficiency, we conducted a joint tuning sweep of the candidate window size passed from the Cross-Encoder (K_{cross}) and the depth of the LLM reranking (K_{llm}).

The results in **(Figure 14)** indicated that the performance saturates quickly. As observed in the tuning logs, smaller window sizes ($K_{cross} = 30$) frequently outperformed or matched larger windows ($K_{cross} = 50+$). This validates the efficiency of the pipeline: the relevant signals are concentrated at the very top. Forcing the LLM to process deeper ranks (e.g., rank 31-100) introduces noise rather than signal. Consequently, we configured the final stage with $K_{cross} = 30$ and $K_{llm} = 10$, balancing maximum precision with computational cost.

5 Final Evaluation and Pipeline Analysis

To validate the efficacy of our proposed architecture, we evaluated the complete multi-stage pipeline on the held-out test set. This split proved to be significantly more challenging than the development sets, characterized by lower absolute recall baselines and higher query ambiguity. Despite these

challenges, the results confirm that a cascaded retrieval approach is essential for solving Tip-of-the-Tongue (ToT) tasks.

5.1 Overall Performance Trajectory

The experimental results demonstrate a consistent, step-by-step improvement in ranking quality as the pipeline progresses from sparse retrieval to LLM powered list-wise reranking. As summarized in **(Table 13)**, no single model is sufficient; rather, each stage contributes a specific layer of refinement.

Table 13: Cumulative performance of the retrieval pipeline on the Test Set. The final GenAI-enhanced stage achieves a 113% improvement over the traditional sparse baseline.

Pipeline Stage	nDCG@10	Success@10
Sparse Baseline (BM25)	0.1738	0.2476
Dense Fusion (RRF)	0.2338	0.3457
Cross-Encoder (MonoT5)	0.3110	0.4244
4. LLM Reranker (Qwen)	**0.3706**	**0.4550**

R@1000	Gain (nDCG)
0.6302	–
0.6302	+34.5%
0.5756	+78.9%
0.5756	**+113.2%**

The final system achieves an nDCG@10 of **0.3706**, representing a **+113% gain** over the sparse baseline. This dramatic lift validates our hypothesis: while lexical matching provides a necessary foundation, only deep semantic interaction (Cross-Encoders) and world-knowledge reasoning (LLMs) can bridge the vocabulary gap inherent in ToT queries.

5.2 Stage-by-Stage Analysis

5.2.1 ***Stage 1: The Sparse Foundation :*** The BM25 baseline, augmented with Mistral-based query rewriting, achieved a Recall@1000 of 0.6302. Compared to the development sets (where recall often exceeded 0.80), this lower ceiling indicates that the test set contains "hard" queries with severe vocabulary mismatches that even generative rewriting could not fully resolve. This established a strict upper bound for retrieval coverage in subsequent stages.

5.2.2 ***Stage 2: Dense Retrieval and The "E5 Factor" :*** The dense retrieval stage acted as a critical recovery mechanism, boosting nDCG@10 to 0.2338. However, the performance distribution among the constituent dense models was highly uneven:

- **ColBERTv2 (0.1361) and Contriever (0.1458):** These models underperformed the sparse baseline. This suggests a "semantic drift" where the models, confused by the abstract nature of the rewritten queries, retrieved documents that were topically related but factually incorrect.
- **E5-Large (0.2253):** The E5 model significantly outperformed its peers, single-handedly driving the success of the dense stage. Its strong instruction-following capabilities likely aligned better with the structure of the rewritten queries.

The Reciprocal Rank Fusion (RRF) successfully combined these divergent signals, producing a final list (0.2338) that outperformed even the best single model (E5), justifying the computational overhead of the ensemble.

5.2.3 ***The Recall Ceiling Phenomenon :*** A notable observation in **(Table 13)** is that Recall@1000 remained constant at 0.6302 from Stage 1 through Stage 2. This is a structural consequence of our "Sparse-First Cascade" design. The dense models were configured to re-rank the top-1000 candidates provided by BM25 rather than retrieving from the full corpus. Consequently, the dense stage recall was mathematically capped by the sparse stage recall. While this limits total coverage, it massively reduces the search space for the dense vectors.

5.2.4 ***Stages 3 and 4: Precision Enhancement :*** The final two stages acted as "Precision Enhancers", tasked with cleaning the noisy candidate list.

- **Cross-Encoder:** The MonoT5 model delivered the single largest jump in the pipeline, increasing nDCG@10 from 0.2338 to 0.3110. It successfully identified relevant documents that had been buried deep in the fusion list and promoted them to the top 10.
- **LLM Reranker:** The 72B parameter model provided the final polish, achieving a further +19% gain over the Cross-Encoder. By leveraging its extensive world knowledge, it rescued hard cases, pushing Success@10 to 0.4550, demonstrating an ability to solve queries that lacked sufficient semantic overlap for the smaller models to detect.

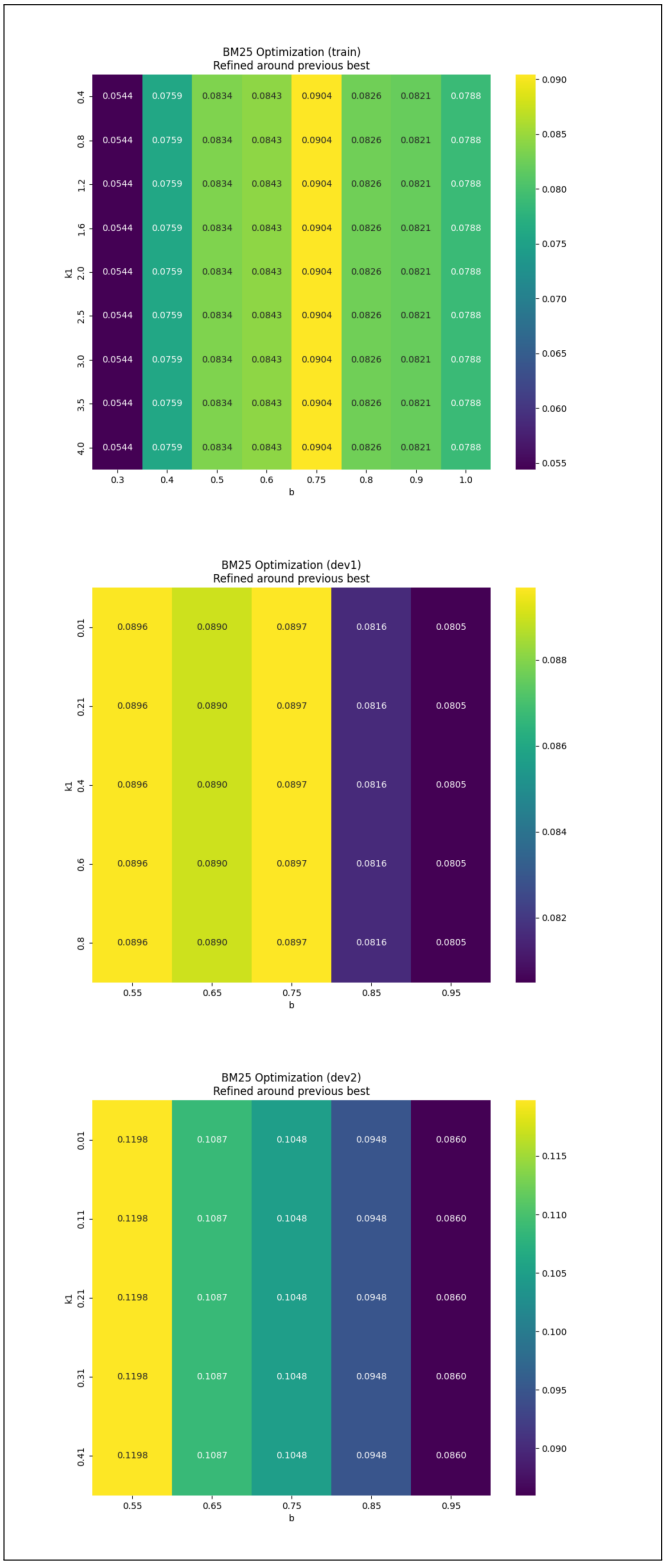

Figure 7: Sequential optimization of BM25 parameters (k_1, b) for Mistral. The search space narrows from the global training grid (left) to refined local grids on dev sets, converging to $k_1 = 0.01$.

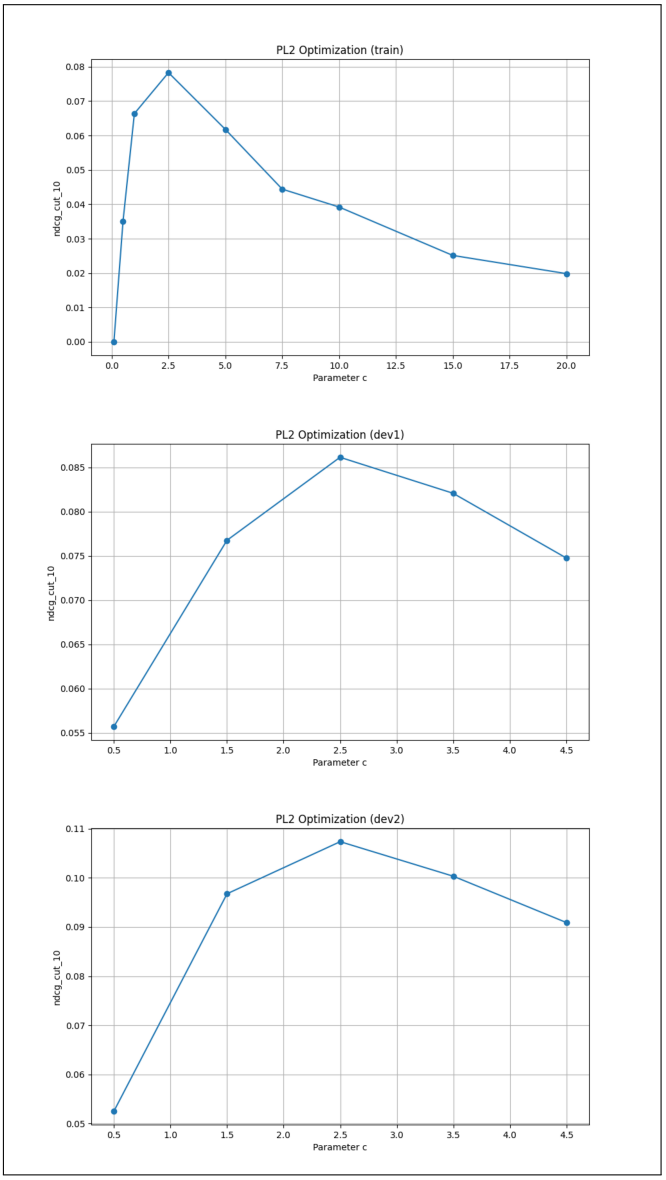

Figure 8: Sequential optimization of the PL2 parameter c. The curve shows a distinct global peak around $c = 2.5$ in the training phase, which remains stable across subsequent development splits.

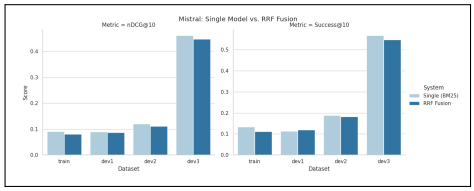

Figure 9: Performance contrast between the single best model (Mistral BM25) and RRF Fusion. The negative values indicate that fusion consistently degrades retrieval quality across all dataset splits.

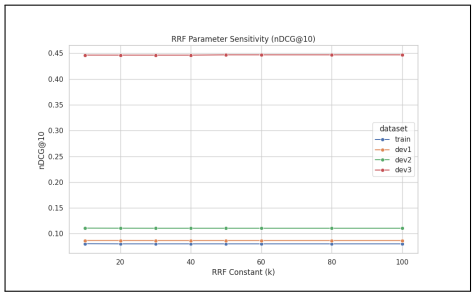

Figure 10: Sensitivity analysis of the RRF constant k. The flat trajectory indicates that hyperparameter tuning cannot recover the performance loss caused by fusing the weaker PL2 signal.

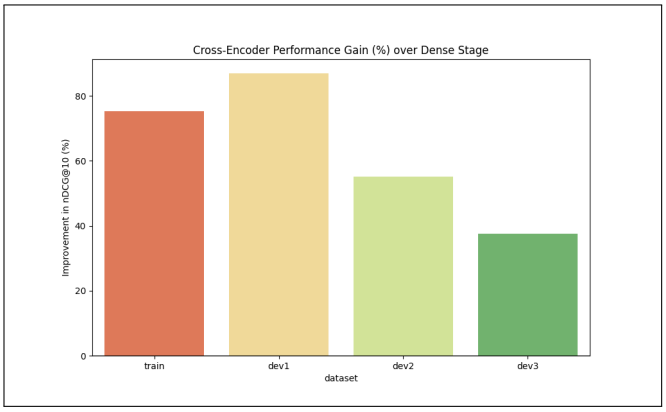

Figure 11: The performance trajectory across pipeline stages. The Cross-Encoder (green) corrects the dip caused by the Dense stage (orange), achieving a new state-of-the-art peak.

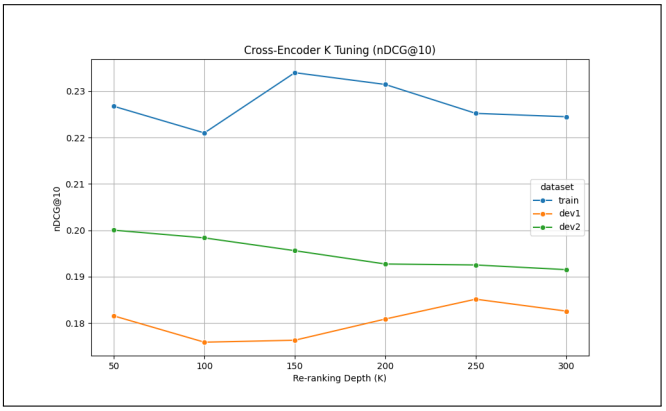

Figure 12: Robust Score of the Cross-Encoder across different candidate depths (K). The performance peaks early at $K = 50$, indicating diminishing returns for deeper re-ranking.

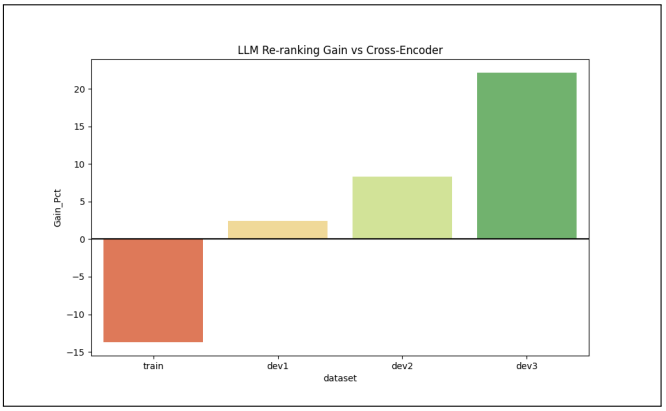

Figure 13: Performance gains across datasets. Note the massive lift on the test set (Dev3), indicating strong generalization, despite regression on the heavily tuned training set.

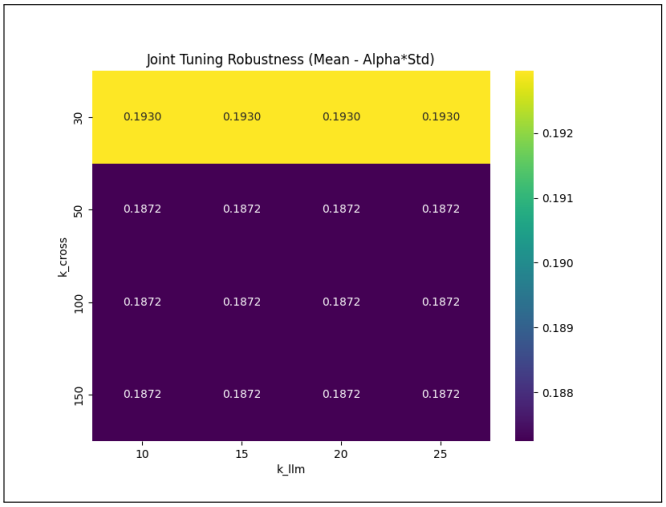

Figure 14: Joint tuning landscape for K_{cross} and K_{llm}. The performance surface is relatively flat, favoring smaller, efficient window sizes.

6 Conclusion

The comprehensive evaluation on the held-out test set confirms that solving the Tip-of-the-Tongue retrieval challenge requires a sophisticated, multi-stage architecture. Our experiments demonstrate that relying solely on traditional sparse retrieval or even standalone dense retrievers is insufficient for the semantic ambiguity inherent in ToT queries.

The proposed pipeline successfully navigates the trade-off between recall and precision. By establishing a stable recall foundation with Mistral-enhanced BM25 and E5-driven dense fusion, we enabled the subsequent discriminative models to focus on ranking quality. The hierarchical refinement strategy-culminating in the application of Qwen-72B resulted in a 113% improvement in nDCG@10 over the baseline. This finding underscores the critical role of Large Language Models not just as generators, but as advanced semantic reasoners within the retrieval loop, capable of bridging the gap between a user's vague recollection and the precise target document.

References

[1] Jaime Arguello, Adam Ferguson, Emery Fine, Bhaskar Mitra, Hamed Zamani, and Fernando Diaz. 2021. Tip of the Tongue Known-Item Retrieval: A Case Study in Movie Identification. In *Proceedings of the 2021 Conference on Human Information Interaction and Retrieval (CHIIR '21)*. ACM, 5–14.

[2] Samarth Bhargav, Georgios Sidiropoulos, and Evangelos Kanoulas. 2022. 'It's on the tip of my tongue': A new dataset for known-item retrieval. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining (WSDM '22)*. ACM, 48–56.

[3] Roger Brown and David McNeill. 1966. The "tip of the tongue" phenomenon. *Journal of Verbal Learning and Verbal Behavior* 5, 4 (1966), 325–337.

[4] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.

[5] Maik Fröbe, Eric Oliver Schmidt, and Matthias Hagen. 2023. A Large-Scale Dataset for Known-Item Question Performance Prediction. In *Proceedings of the QPP++ 2023 Workshop at ECIR (CEUR Workshop Proceedings, Vol. 3366)*. 13–19.

[6] Claudia Hauff, Matthias Hagen, Anna Beyer, and Benno Stein. 2012. Towards realistic known-item topics for the ClueWeb. In *Proceedings of the 4th Information Interaction in Context Symposium (IIiX '12)*. ACM, 274–277.

[7] Yifan He, To Eun Kim, Fernando Diaz, Jaime Arguello, and Bhaskar Mitra. 2025. Tip of the Tongue Query Elicitation for Simulated Evaluation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 3398–3407.

[8] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Towards Unsupervised Dense Information Retrieval with Contrastive Learning. *arXiv preprint arXiv:2112.09118* (2021).

[9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[10] Kevin Lin, Kyle Lo, Joseph E Gonzalez, and Dan Klein. 2023. Decomposing Complex Queries for Tip-of-the-tongue Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 5521–5533.

[11] OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, Aleksander Mądry, Alex Baker-Whitcomb, Alex Beutel, Alex Borzunov, Alex Carney, Alex Chow, Alex Kirillov, Alex Nichol, Alex Paino, Alex Renzin, Alex Tachard Passos, Alexander Kirillov, Alexi Christakis, Alexis Conneau, Ali Kamali, Allan Jabri, Allison Moyer, Allison Tam, Amadou Crookes, Amin Tootoochian, Amin Tootoonchian, Ananya Kumar, Andrea Vallone, Andrej Karpathy, Andrew Braunstein, Andrew Cann, Andrew Codispoti, Andrew Galu, Andrew Kondrich, Andrew Tulloch, Andrey Mishchenko, Angela Baek, Angela Jiang, Antoine Pelisse, Antonia Woodford, Anuj Gosalia, Arka Dhar, Ashley Pantuliano, Avi Nayak, Avital Oliver, Barret Zoph, Behrooz Ghorbani, Ben Leimberger, Ben Rossen, Ben Sokolowsky, Ben Wang, Benjamin Zweig, Beth Hoover, Blake Samic, Bob McGrew, Bobby Spero, Bogo Giertler, Bowen Cheng, Brad Lightcap, Brandon Walkin, Brendan Quinn, Brian Guarraci, Brian Hsu, Bright Kellogg, Brydon Eastman, Camillo Lugaresi, Carroll Wainwright, Cary Bassin, Cary Hudson, Casey Chu, Chad Nelson, Chak Li, Chan Jun Shern, Channing Conger, Charlotte Barette, Chelsea Voss, Chen Ding, Cheng Lu, Chong Zhang, Chris Beaumont, Chris Hallacy, Chris Koch, Christian Gibson, Christina Kim, Christine Choi, Christine McLeavey, Christopher Hesse, Claudia Fischer, Clemens Winter, Coley Czarnecki, Colin Jarvis, Colin Wei, Constantin Koumouzelis, Dane Sherburn, Daniel Kappler, Daniel Levin, Daniel Levy, David Carr, David Farhi, David Mely, David Robinson, David Sasaki, Denny Jin, Dev Valladares, Dimitris Tsipras, Doug Li, Duc Phong Nguyen, Duncan Findlay, Edede Oiwoh, Edmund Wong, Ehsan Asdar, Elizabeth Proehl, Elizabeth Yang, Eric Antonow, Eric Kramer, Eric Peterson, Eric Sigler, Eric Wallace, Eugene Brevdo, Evan Mays, Farzad Khorasani, Felipe Petroski Such, Filippo Raso, Francis Zhang, Fred von Lohmann, Freddie Sulit, Gabriel Goh, Gene Oden, Geoff Salmon, Giulio Starace, Greg Brockman, Hadi Salman, Haiming Bao, Haitang Hu, Hannah Wong, Haoyu Wang, Heather Schmidt, Heather Whitney, Heewoo Jun, Hendrik Kirchner, Henrique Ponde de Oliveira Pinto, Hongyu Ren, Huiwen Chang, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian O'Connell, Ian Osband, Ian Silber, Ian Sohl, Ibrahim Okuyucu, Ikai Lan, Ilya Kostrikov, Ilya Sutskever, Ingmar Kanitscheider, Ishaan Gulrajani, Jacob Coxon, Jacob Menick, Jakub Pachocki, James Aung, James Betker, James Crooks, James Lennon, Jamie Kiros, Jan Leike, Jane Park, Jason Kwon, Jason Phang, Jason Teplitz, Jason Wei, Jason Wolfe, Jay Chen, Jeff Harris, Jenia Varavva, Jessica Gan Lee, Jessica Shieh, Ji Lin, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joanne Jang, Joaquin Quinonero Candela, Joe Beutler, Joe Landers, Joel Parish, Johannes Heidecke, John Schulman, Jonathan Lachman, Jonathan McKay, Jonathan Uesato, Jonathan Ward, Jong Wook Kim, Joost Huizinga, Jordan Sitkin, Jos Kraaijeveld, Josh Gross, Josh Kaplan, Josh Snyder, Joshua Achiam, Joy Jiao, Joyce Lee, Juntang Zhuang, Justyn Harriman, Kai Fricke, Kai Hayashi, Karan Singhal, Katy Shi, Kavin Karthik, Kayla Wood, Kendra Rimbach, Kenny Hsu, Kenny Nguyen, Keren Gu-Lemberg, Kevin Button, Kevin Liu, Kiel Howe, Krithika Muthukumar, Kyle Luther, Lama Ahmad, Larry Kai, Lauren Itow, Lauren Workman, Leher Pathak, Leo Chen, Li Jing, Lia Guy, Liam Fedus, Liang Zhou, Lien Mamitsuka, Lilian Weng, Lindsay McCallum, Lindsey Held, Long Ouyang, Louis Feuvrier, Lu Zhang, Lukas Kondraciuk, Lukasz Kaiser, Luke Hewitt, Luke Metz, Lyric Doshi, Mada Aflak, Maddie Simens, Madelaine Boyd, Madeleine Thompson, Marat Dukhan, Mark Chen, Mark Gray, Mark Hudnall, Marvin Zhang, Marwan Aljubeh, Mateusz Litwin, Matthew Zeng, Max

Johnson, Maya Shetty, Mayank Gupta, Meghan Shah, Mehmet Yatbaz, Meng Jia Yang, Mengchao Zhong, Mia Glaese, Mianna Chen, Michael Janner, Michael Lampe, Michael Petrov, Michael Wu, Michele Wang, Michelle Fradin, Michelle Pokrass, Miguel Castro, Miguel Oom Temudo de Castro, Mikhail Pavlov, Miles Brundage, Miles Wang, Minal Khan, Mira Murati, Mo Bavarian, Molly Lin, Murat Yesildal, Nacho Soto, Natalia Gimelshein, Natalie Cone, Natalie Staudacher, Natalie Summers, Natan LaFontaine, Neil Chowdhury, Nick Ryder, Nick Stathas, Nick Turley, Nik Tezak, Niko Felix, Nithanth Kudige, Nitish Keskar, Noah Deutsch, Noel Bundick, Nora Puckett, Ofir Nachum, Ola Okelola, Oleg Boiko, Oleg Murk, Oliver Jaffe, Olivia Watkins, Olivier Godement, Owen Campbell-Moore, Patrick Chao, Paul McMillan, Pavel Belov, Peng Su, Peter Bak, Peter Bakkum, Peter Deng, Peter Dolan, Peter Hoeschele, Peter Welinder, Phil Tillet, Philip Pronin, Philippe Tillet, Prafulla Dhariwal, Qiming Yuan, Rachel Dias, Rachel Lim, Rahul Arora, Rajan Troll, Randall Lin, Rapha Gontijo Lopes, Raul Puri, Reah Miyara, Reimar Leike, Renaud Gaubert, Reza Zamani, Ricky Wang, Rob Donnelly, Rob Honsby, Rocky Smith, Rohan Sahai, Rohit Ramchandani, Romain Huet, Rory Carmichael, Rowan Zellers, Roy Chen, Ruby Chen, Ruslan Nigmatullin, Ryan Cheu, Saachi Jain, Sam Altman, Sam Schoenholz, Sam Toizer, Samuel Miserendino, Sandhini Agarwal, Sara Culver, Scott Ethersmith, Scott Gray, Sean Grove, Sean Metzger, Shamez Hermani, Shantanu Jain, Shengjia Zhao, Sherwin Wu, Shino Jomoto, Shirong Wu, Shuaiqi, Xia, Sonia Phene, Spencer Papay, Srinivas Narayanan, Steve Coffey, Steve Lee, Stewart Hall, Suchir Balaji, Tal Broda, Tal Stramer, Tao Xu, Tarun Gogineni, Taya Christianson, Ted Sanders, Tejal Patwardhan, Thomas Cunninghman, Thomas Degry, Thomas Dimson, Thomas Raoux, Thomas Shadwell, Tianhao Zheng, Todd Underwood, Todor Markov, Toki Sherbakov, Tom Rubin, Tom Stasi, Tomer Kaftan, Tristan Heywood, Troy Peterson, Tyce Walters, Tyna Eloundou, Valerie Qi, Veit Moeller, Vinnie Monaco, Vishal Kuo, Vlad Fomenko, Wayne Chang, Weiyi Zheng, Wenda Zhou, Wesam Manassra, Will Sheu, Wojciech Zaremba, Yash Patil, Yilei Qian, Yongjik Kim, Youlong Cheng, Yu Zhang, Yuchen He, Yuchen Zhang, Yujia Jin, Yunxing Dai, and Yury Malkov. 2024. GPT-4o System Card. arXiv:2410.21276 [cs.CL] https://arxiv.org/abs/2410.21276

[12] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

[13] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] https://arxiv.org/abs/2412.15115

[14] TREC-TOT Organizers. 2025. *TREC-TOT 2025 Guidelines.* https://trec-tot.github.io/guidelines

[15] Sky CH Wang, Darshan Deshpande, Smaranda Muresan, Anand Kannappan, and Rebecca Qian. 2025. Browsing Lost Unformed Recollections: A Benchmark for Tip-of-the-Tongue Search and Reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL).* Association for Computational Linguistics, 8317–8331.